Instituto Politécnico Nacional

Escuela Superior de Cómputo

Unidad Académica:

Sistemas Distribuidos

Profesor:

Carreto Arellano Chadwick

Práctica 04: "Escalando al modelo múltiples clientes y múltiples servidores"

Alumna:

Maldonado Bedolla Daniela del Carmen

Grupo:

7CM1

12 de marzo de 2026

Índice

INTRODUCCIÓN **3**

DESARROLLO **4**

PROBLEMÁTICA 4
PROPUESTA DE SOLUCIÓN 4
IMPLEMENTACIÓN 7
Generalidades *7*
Servidor *8*
Cliente *10*
Coordinador *12*

EJECUCIÓN Y RESULTADOS **14**

EJECUCIÓN DE LA SIMULACIÓN 14

RESULTADOS **20**

SERVIDOR S1 20
Resumern del servidor *20*
Clientes del servidor *21*
SERVIDOR S2 21
Resumen del servidor *22*
Cliente del servidor *22*

COMPARACIÓN DE RESULTADOS CON PRÁCTICAS ANTERIORES **23**

CONCLUSIONES **25**

ANEXO **26**

CÓDIGO DEL COORDINADOR 26
CÓDIGO DEL SERVIDOR 27
CÓDIGO DE CLIENTE 28

INTRODUCCIÓN

La progresión de sistemas concurrentes hacia arquitecturas distribuidas y escalables constituye un eje central en el desarrollo de software moderno. En la práctica inicial, el problema de venta de boletos se abordó en un entorno local con múltiples hilos dentro de un mismo proceso, lo que permitió estudiar condiciones de carrera, exclusión mutua y consistencia sobre memoria compartida. Posteriormente, al migrar al modelo cliente/servidor, la administración del recurso crítico se centralizó en el servidor, incorporando comunicación por sockets y sincronización basada en mensajes en lugar de acceso directo a estructuras compartidas.

En esta etapa, la arquitectura evoluciona hacia un escenario de **múltiples clientes y múltiples servidores**, con un enfoque explícito en escalabilidad y coordinación distribuida. Cada servidor administra su propia venta (por ejemplo, distintas fechas de un mismo evento), mientras un **coordinador global** orquesta el arranque conjunto de las ventas mediante una barrera de sincronización inter-servidor. De este modo, el inicio ya no depende solo del estado local de un servidor, sino de una condición global que integra afluencia de clientes y disponibilidad de todos los nodos participantes.

Para sostener la integridad del sistema bajo alta concurrencia, se mantienen y extienden mecanismos de control: bloqueo por zonas, reservas temporales con expiración, validación de propiedad de reserva, confirmación atómica de compra y cierre ordenado de conexiones entre procesos. Además, se incorpora observabilidad en tiempo real con paneles visuales para monitorear estados de servidores, progreso de conexión de clientes y transición de fases de la venta, complementado por métricas de desempeño en cada nodo.

En conjunto, esta práctica consolida una base de sincronización distribuida sobre recursos finitos, dejando abierta la posibilidad a etapas futuras orientadas a servicios, objetos distribuidos y arquitecturas más avanzadas de coordinación y tolerancia a fallos.

Desarrollo

Problemática

En la versión original del modelo cliente/servidor, la dinámica era una sola venta centralizada: una “flecha” de tráfico desde múltiples plataformas cliente hacia un único servidor que administraba el inventario. En ese esquema, muchos clientes actuaban como puntos de acceso con alta afluencia de compradores concurrentes, y todos competían por un conjunto finito de asientos controlado por una sola autoridad de estado.

Al escalar el escenario, el problema se extiende a la venta de **N fechas en paralelo**. Esto implica **N servidores de venta** (uno por fecha) y múltiples clientes que generan afluencia para cada servidor según la fecha objetivo. En este contexto, los retos ya no son solo de concurrencia local por servidor, sino también de coordinación global entre servidores para iniciar y operar ventas simultáneas sin inconsistencias.

Los riesgos principales pasan a ser:

- Competencia intensa por asientos dentro de cada servidor (doble intento de reserva/compra sobre el mismo recurso).
- Desfase entre estado percibido por clientes y estado real del servidor por latencia o reintentos.
- Reservas no confirmadas que pueden “atorar” asientos si no se liberan automáticamente.
- Alta tasa de intentos fallidos en la etapa final de cada venta (pocos asientos restantes).
- Desalineación de inicio entre servidores si no existe sincronización global para arrancar todas las fechas al mismo tiempo.
- Persistencia de conexiones abiertas al finalizar, afectando cierre limpio de procesos distribuidos.

En síntesis, la problemática evoluciona de “muchos hilos contra un inventario” a “muchos clientes y muchos servidores coordinados”, donde se debe garantizar consistencia por venta y sincronización del sistema completo.

Propuesta de solución

La solución se estructuró en una arquitectura multi-cliente / multi-servidor con coordinación central:

- Cada servidor administra una venta independiente (una fecha) y mantiene autoridad total sobre sus asientos.
- Cada cliente (punto de acceso) se dirige explícitamente a un servidor/fecha (`sale_id`) y genera afluencia mediante hilos compradores.
- Un coordinador central registra servidores y clientes conectados por fecha, y sincroniza el arranque global con señal `GLOBAL_START`.
- El servidor conserva control estricto de estados de asiento (`FREE`, `RESERVED`, `SOLD`) y exclusión mutua para evitar doble venta.
- Las reservas son temporales (TTL); si no se confirman, se liberan automáticamente y regresan a disponibilidad.
- La compra solo la puede confirmar el mismo comprador que creó la reserva (validación de propiedad).
- Cada venta cierra por condición de término (agotamiento, fin de clientes o inviabilidad de demanda elegible), manteniendo consistencia de conteos.
- Se incorporó visualización en tiempo real:
    - en servidor: progreso de la venta por asientos;
    - en coordinador: estado de servidores y afluencia de clientes por fecha.
- Se reforzó el cierre lógico de conexiones:
    - notificación explícita de desconexión servidor-coordinador;
    - cierre ordenado de sockets/streams en cliente, servidor y coordinador;
    - configuración de hilos daemon para evitar bloqueos en apagado.

Con este enfoque se obtiene escalabilidad horizontal por fecha, sincronización global de arranque, integridad transaccional por servidor y finalización limpia de ejecución en todo el sistema distribuido.

usa un modelo cliente/servidor donde el servidor es la autoridad total del sistema:

- El cliente crea los 2,000 hilos compradores y cada hilo manda solicitudes por socket.
- El servidor valida todo: solicitud de asiento, apartado y compra final.
- Los asientos se manejan con estados (FREE, RESERVED, SOLD) y control de exclusión para evitar doble venta.
- El apartado tiene tiempo límite (TTL); si no se confirma a tiempo, el asiento se libera automáticamente.

- La compra solo se permite al mismo comprador que hizo la reserva (validación de dueño de reserva).
- Al llegar a 1,000 vendidos, el servidor marca venta terminada y deja de aceptar nuevas compras.

Además, se agregó visualización en el servidor y métricas de tiempo para observar el comportamiento del sistema.
Con este enfoque se logra que no haya ventas duplicadas, que el conteo final sea correcto y que el cierre de ejecución sea limpio tanto en servidor como en cliente.

Implementación

Generalidades

La implementación actual extiende el problema original hacia un escenario de N ventas simultáneas. En lugar de tener una sola venta centralizada, ahora el sistema permite ejecutar múltiples servidores de boletos en paralelo, donde cada uno representa una venta independiente, por ejemplo, distintas fechas de un mismo concierto. Cada venta mantiene su propio inventario, su propia concurrencia interna y sus propios clientes asociados, pero puede coordinarse con otras ventas mediante un proceso adicional.

A nivel visual, el sistema incorpora dos tipos de interfaz. Por un lado, cada servidor muestra su propia simulación de asientos; por otro, el coordinador genera un grid dinámico cuyo tamaño depende del número de servidores registrados. En ese grid se observan los estados globales de cada venta y la afluencia de clientes por servidor. Esto permite ver el sistema no solo como una venta aislada, sino como un conjunto de ventas concurrentes que avanzan en paralelo.

En la versión implementada, el foro ya no se modela como una matriz de 25 x 40, sino como una matriz de 30 filas x 50 columnas, para un total de 1,500 asientos. Además, los asientos no se manejan únicamente como disponible o vendido, sino mediante tres estados: FREE, RESERVED y SOLD. Esto fue necesario para soportar reservas temporales, confirmación posterior de compra y liberación automática en caso de expiración.

También se agregaron secciones y tipos de comprador. El foro se divide en tres zonas:

- Platino
- Preferente
- Normal

Y los compradores se clasifican por tipo:

- platino
- preferente
- normal

Cada tipo de comprador tiene reglas de acceso distintas sobre las zonas permitidas. Esto introduce una lógica adicional de elegibilidad, ya que no todos los compradores pueden adquirir cualquier asiento. En consecuencia, el sistema no solo resuelve concurrencia, sino también restricciones de acceso, administración de demanda y cierre anticipado cuando los asientos restantes ya no pueden ser comprados por los tipos de clientes conectados.

Servidor

El servidor concentra la lógica crítica del sistema: administra inventario, aplica reglas de acceso, sincroniza compras concurrentes, mantiene el estado de cada asiento y define cuándo inicia y termina una venta. Es la autoridad total de cada venta. Controla el estado real de los asientos, valida reservas y compras, mantiene métricas y define el cierre de la venta. A continuación, se muestran las partes críticas de su implementación.

1. Definición del foro, zonas y reglas de acceso

```
FILAS = 30
COLUMNAS = 50
TOTAL_ASIENTOS = FILAS * COLUMNAS
RESERVA_TTL_SEGUNDOS = 1.0
SECTION_GAP_ROWS = 2
SECTION_LABEL_ROWS = 1

ZONA_PLATINO = "PLATINO"
ZONA_PREFERENTE = "PREFERENTE"
ZONA_NORMAL = "NORMAL"

TIPO_PLATINO = "platino"
TIPO_PREFERENTE = "preferente"
TIPO_NORMAL = "normal"

ALLOWED_ZONES_BY_TYPE = {
    TIPO_PLATINO: [ZONA_PLATINO, ZONA_PREFERENTE, ZONA_NORMAL],
    TIPO_PREFERENTE: [ZONA_PREFERENTE, ZONA_NORMAL],
    TIPO_NORMAL: [ZONA_NORMAL],
}
```

- *FILAS y COLUMNAS definen el tamaño de la matriz del foro.*
- *TOTAL_ASIENTOS permite calcular rápidamente capacidad total, métricas y cierre por agotamiento.*
- *ZONA_PLATINO, ZONA_PREFERENTE y ZONA_NORMAL nombran las secciones lógicas del recinto.*
- *ALLOWED_ZONES_BY_TYPE establece qué zonas puede comprar cada tipo de cliente.*

- *Esta parte es clave porque transforma el problema de concurrencia simple en un problema con restricciones de elegibilidad.*

2. Estado centralizado y sincronización interna

```python
class TicketState:
    def __init__(self):
        self.meta_lock = threading.Lock()
        self.terminal_lock = threading.Lock()

        self.zone_locks = {
            ZONA_PLATINO: threading.Lock(),
            ZONA_PREFERENTE: threading.Lock(),
            ZONA_NORMAL: threading.Lock(),
        }

        self.zone_free_seats = build_zone_seats()
        self.seat_status = [["FREE" for _ in range(COLUMNAS)] for _ in

        self.reservations_by_zone = {
            ZONA_PLATINO: {},
            ZONA_PREFERENTE: {},
            ZONA_NORMAL: {},
        }
```

- *zone_locks implementa exclusión mutua por zona, permitiendo concurrencia controlada sin bloquear toda la venta.*
- *zone_free_seats mantiene el conjunto de asientos libres por sección.*
- *seat_status guarda el estado visible de cada asiento: FREE, RESERVED o SOLD.*
- *reservations_by_zone almacena reservas activas por zona.*
- *Esta organización evita doble venta y permite escalar mejor que un solo candado global.*

3. Reserva temporal y confirmación de compra

```python
    self.zone_free_seats[zone].remove(seat)
    self.seat_status[row][col] = "RESERVED"
    self.reservations_by_zone[zone][reservation_id] = {
        "buyer_id": str(buyer_id),
        "buyer_type": (buyer_type or TIPO_NORMAL).lower(),
        "seat": seat,
        "zone": zone,
        "expires_at": time.monotonic() + RESERVA_TTL_SEGUNDOS,
        "request_id": request_id,
    }
```

- *reservation_id identifica de forma única cada apartado.*
- *buyer_id se usa para validar que solo el dueño pueda concretar la compra.*
- *expires_at define el TTL de la reserva.*
- *seat y zone permiten restaurar o vender el recurso exacto.*
- *seat_status cambia primero a RESERVED y luego a SOLD si la compra se confirma.*
- *Esto evita que un cliente retenga indefinidamente un asiento y mejora la consistencia del inventario.*

4. Liberación automática de reservas expiradas

```python
def cleanup_expired_reservations(ticket_state):
    while not ticket_state.sold_out_event.is_set():
        if ticket_state.sales_open_event.is_set():
            for zone in (ZONA_PLATINO, ZONA_PREFERENTE, ZONA_NORMAL):
                zone_lock = ticket_state.zone_locks[zone]
                acquired = zone_lock.acquire(timeout=0.05)
                if acquired:
                    try:
                        ticket_state._cleanup_expired_zone_locked(zone
                    finally:
                        zone_lock.release()
        time.sleep(0.25)
```

- *cleanup_expired_reservations corre en un hilo aparte.*
- *_cleanup_expired_zone_locked revisa reservas vencidas y devuelve los asientos a FREE.*
- *sold_out_event sirve como condición de parada del hilo.*
- *Este mecanismo corrige el problema de asientos atascados en naranja y garantiza liberación periódica aun si ya no llegan nuevas solicitudes.*

Cliente

El cliente representa un punto de acceso. Desde él se generan múltiples compradores como hilos, cada uno compitiendo por conseguir boletos. Su función principal no es administrar inventario, sino producir afluencia concurrente hacia el servidor correcto.

Este funciona como una plataforma de acceso que concentra afluencia. No decide el estado del sistema, sino que genera carga concurrente mediante muchos compradores que operan sobre el servidor correspondiente a una venta específica.

1. Identificación del servidor objetivo y de la venta

```python
def main():
    global sales_start_ts, sales_end_ts

    args = parse_args()
    normalized_type = normalize_client_type(args.client_type)

    if args.buyers <= 0:
        raise ValueError("buyers debe ser mayor que 0")

    client_id = args.client_id or f"{args.client_type.upper()}-{str(uuid.uuid4())[:6]}"
    sale_id = args.sale_id or f"{args.host}:{args.port}"
```

- *sale_id identifica explícitamente la venta a la que el cliente se dirige.*
- *host y port determinan el servidor físico de destino.*
- *Esto es importante porque ahora existen múltiples ventas simultáneas y el cliente ya no puede asumirse ligado a un único servidor global.*

2. Registro, READY y espera de START

```python
send_control_message(
    sock_file,
    {
        "type": "REGISTER",
        "request_id": str(uuid.uuid4()),
        "client_id": client_id,
        "client_type": client_type,
        "buyers": buyers_count,
    },
)
```

```python
send_control_message(
    sock_file,
    {
        "type": "READY",
        "request_id": str(uuid.uuid4()),
        "client_id": client_id,
    },
)
```

- *REGISTER informa al servidor que un nuevo punto de acceso se conectó.*
- *READY indica que el cliente terminó su preparación local.*
- *START desbloquea el inicio real de la venta.*
- *sock_file se usa como canal de control persistente durante la fase de sincronización.*
- *Este flujo permite separar claramente conexión, preparación e inicio efectivo.*

3. Generación de compradores concurrentes

```python

    for buyer_number in range(1, args.buyers + 1):
        thread = threading.Thread(
            target=buyer_worker,
            args=(buyer_number, args.host, args.port, client_id, normalized_type),
            daemon=False,
        )
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)
```

- *args.buyers define cuántos hilos compradores se crearán.*
- *buyer_worker representa a un comprador individual.*
- *threads permite esperar el cierre de todos los compradores antes de finalizar el cliente.*
- *Esta parte materializa la afluencia real hacia el servidor.*

4. Ciclo de compra del comprador

```python
request_payload = {
    "type": "REQUEST_TICKET",
    "request_id": str(uuid.uuid4()),
    "buyer_id": buyer_id,
    "buyer_type": client_type,
}
```

```python
purchase_payload = {
    "type": "PURCHASE",
    "request_id": str(uuid.uuid4()),
    "buyer_id": buyer_id,
    "reservation_id": reservation_id,
}
```

- *REQUEST_TICKET intenta apartar un asiento permitido para ese tipo de comprador.*
- *PURCHASE intenta confirmar la reserva obtenida.*
- *reservation_id enlaza ambas operaciones.*
- *sold_out_event sirve para frenar todos los hilos cuando la venta termina.*
- *El flujo modela el comportamiento competitivo de múltiples compradores independientes.*

Coordinador

El coordinador fue agregado para sincronizar múltiples ventas y ofrecer observabilidad del sistema. Aunque la sincronización global podría rediseñarse de otra manera en el futuro, su implementación actual permite coordinar el arranque conjunto de varias ventas y visualizar el estado global de la arquitectura.

Este no vende boletos ni administra asientos; su función es sincronizar el inicio global de múltiples ventas y ofrecer una vista unificada del sistema distribuido mientras cada servidor mantiene la autoridad local de su propia fecha.

1. Registro de servidores y grid dinámico

```
slot_to_sale = {}
slot_of_sale = [None for _ in range(expected_servers)]
sale_to_slot = {}
```

```
result, start_now = self.server.state.register_server(
    sale_id,
    self,
    expected_clients,
    server_host,
    server_port,
)
if result is None:
    self.send_json(
        {
            "type": "ERROR",
            "code": "capacity_reached",
```

- *expected_servers define cuántos servidores se esperan en total.*
- *slot_to_sale asocia cada columna del grid con un servidor registrado.*
- *register_server reserva el espacio lógico de ese servidor y actualiza su estado visual.*
- *Esto permite construir una vista global dinámica del sistema sin fijar de antemano nombres o posiciones rígidas.*

2. Registro de clientes y afluencia por venta

```
connected, expected, total_buyers, start_now = self.server.state.register_client_connected(
    sale_id,
    client_id,
    buyers_count,
)
self.send_json(
    {
        "type": "CLIENT_CONNECTED_ACK",
        "sale_id": sale_id,
        "client_id": client_id,
        "connected_clients": connected,
        "expected_clients": expected,
        "total_buyer_threads": total_buyers,
    }
)
```

- *CLIENT_CONNECTED permite que el coordinador sepa cuántos clientes llegaron a cada venta.*
- *sale_id indica a qué servidor corresponde el cliente.*
- *buyers informa la afluencia en términos de hilos compradores.*
- *Con esto, el coordinador deja de ser solo un visor de servidores y también representa la presión de clientes sobre cada venta.*

3. Arranque global sincronizado

```
if start_now:
    print("[Coordinador] Condición global cumplida. Enviando GLOBAL_START.")
    self.server.state.broadcast_global_start()
continue
```

- *start_now se activa cuando todos los servidores requeridos están registrados y todos los clientes esperados ya se conectaron.*
- *broadcast_global_start envía la misma señal a todos los servidores.*
- *Esto implementa una barrera distribuida de arranque.*
- *Su valor principal es garantizar simultaneidad entre múltiples ventas, no solo dentro de un servidor aislado.*

4. Visualización de estados globales

```
state_text = {
    STATUS_GRAY: "Sin conexión",
    STATUS_BLUE: "Conectado",
    STATUS_ORANGE: "Venta en proceso",
    STATUS_GREEN: "Venta finalizada",
}.get(status, "Sin conexión")
```

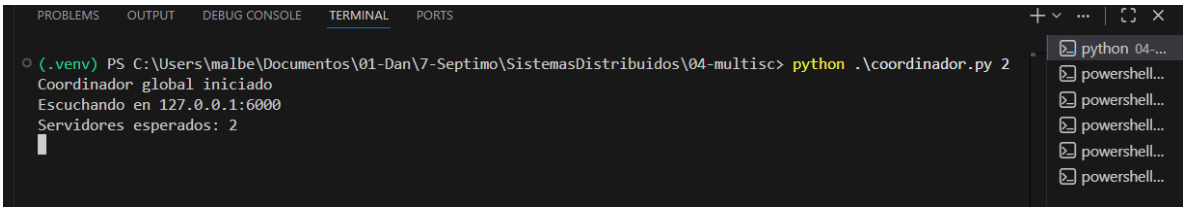
- *STATUS_GRAY indica que ese servidor aún no se registra.*
- *STATUS_BLUE indica que ya está conectado y esperando.*
- *STATUS_ORANGE indica que la venta ya inició.*
- *STATUS_GREEN indica que el proceso de venta terminó.*
- *Esta codificación visual vuelve observable la sincronización global y el avance de múltiples ventas en paralelo.*

Ejecución y Resultados

Ejecución de la simulación

Para validar la arquitectura de múltiples clientes y múltiples servidores, se definió un escenario de prueba con un coordinador central y dos servidores de venta simultáneos. En esta configuración, el coordinador espera el registro de **2 servidores**; el servidor **S1** se configura para esperar **2 clientes**, mientras que el servidor **S2** se configura para esperar **1 cliente**. El objetivo de esta prueba es observar el comportamiento del sistema completo desde el arranque, el registro de servidores, la afluencia de clientes, la sincronización global y el inicio simultáneo de ambas ventas.

1. Inicio del coordinador



La ejecución comienza con el coordinador global. Al arrancar, este crea una ventana con un grid dinámico de acuerdo con la cantidad de servidores esperados. En este caso, se generan dos columnas: una para **Servidor 1** y otra para **Servidor 2**. Inicialmente, ambos servidores aparecen en color **gris**, indicando que todavía no existe conexión registrada. En esta fase, el coordinador únicamente se encuentra a la espera de que los servidores se anuncien formalmente.

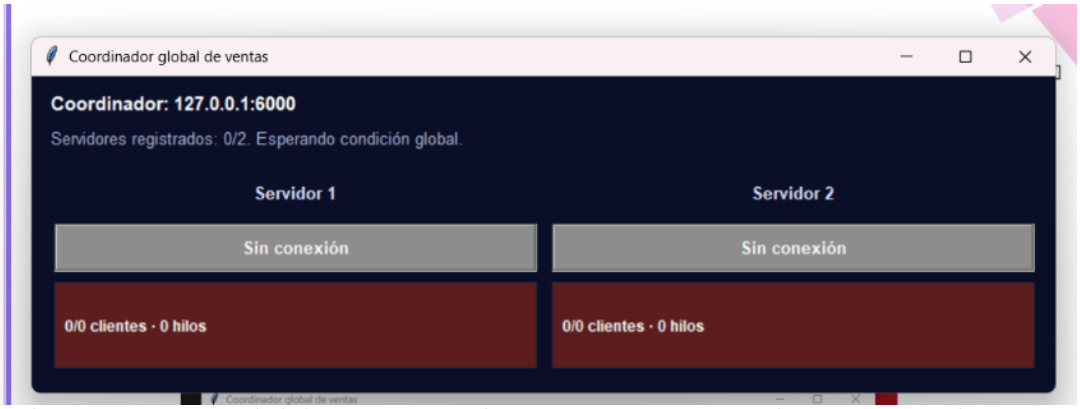


Figura 1. Estado inicial del coordinador antes del registro de servidores.

2. Registro de servidores en el coordinador

Posteriormente, se inician los servidores **S1** y **S2**. Cuando cada uno establece su conexión con el coordinador y envía su mensaje de registro, el coordinador les asigna un espacio en el grid y actualiza su estado visual. El color del recuadro cambia de **gris** a **azul**, lo que significa que el servidor ya está conectado, identificado y listo para recibir afluencia de clientes, aunque la venta todavía no puede comenzar.

En este momento también queda asociado cada servidor con su cantidad esperada de clientes:

- **S1** espera 2 clientes.

```
(.venv) PS C:\Users\malbe\Documentos\01-Dan\7-Septimo\SistemasDistribuidos\04-multisc> python .\servidor.py 2 --po
rt 5001 --sale-id S1 --coordinator-host 127.0.0.1 --coordinator-port 6000
```

- **S2** espera 1 cliente.

```
(.venv) PS C:\Users\malbe\Documentos\01-Dan\7-Septimo\SistemasDistribuidos\04-multisc> python .\servidor.py 1 --po
rt 5002 --sale-id S2 --coordinator-host 127.0.0.1 --coordinator-port 6000
```

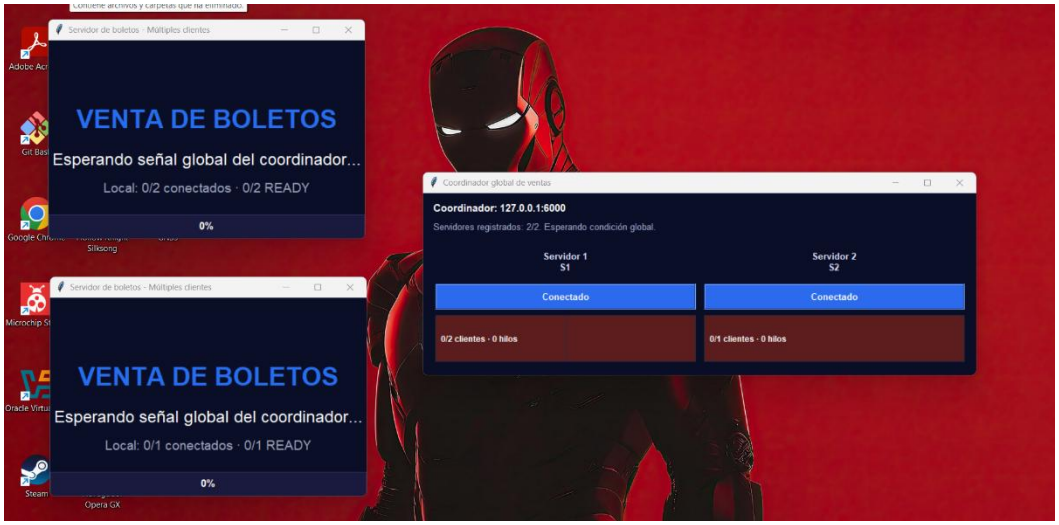


Figura 2. Coordinador con servidores registrados y en estado de espera.

3. Conexión de clientes hacia cada servidor

Una vez activos los servidores, se inician los clientes. Cada cliente se dirige explícitamente a una venta específica mediante su identificador sale_id. En esta prueba:

- Dos clientes se conectan hacia **S1**.

- Un cliente se conecta hacia **S2**.

Cada vez que un cliente se registra ante el coordinador, la tercera fila del grid se actualiza. Esta fila representa la afluencia hacia cada servidor. El rectángulo correspondiente comienza en rojo y se va llenando en verde conforme llegan los clientes esperados. Además, se muestra el conteo de clientes conectados y la cantidad total de hilos compradores generados por cada cliente.

Así, el coordinador no solo verifica que un servidor existe, sino también que ya cuenta con todos los puntos de acceso necesarios para considerar lista su venta local.

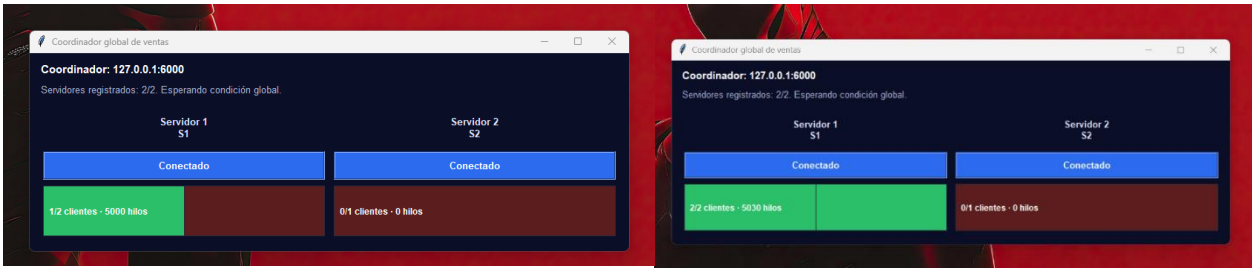


Figura 3. Progreso de conexión de clientes hacia S1 y S2.

4. Cumplimiento de la condición global de inicio

La condición de arranque global se cumple únicamente cuando:

- todos los servidores esperados ya se registraron en el coordinador, y
- cada servidor recibió la cantidad de clientes que tenía configurada.

En el caso de esta prueba, la condición global se satisface cuando:

- **S1** ya tiene conectados sus 2 clientes,
- **S2** ya tiene conectado su único cliente.

En cuanto esta condición se cumple, el coordinador emite la señal **GLOBAL_START** a todos los servidores registrados. En ese instante, los recuadros de estado de los servidores cambian de **azul** a **naranja**, indicando que la fase de venta ya fue habilitada y que ambos nodos deben comenzar de manera sincronizada.

Figura 4. Coordinador al momento de emitir la señal GLOBAL_START.

5. Inicio simultáneo del countdown en los servidores

Después de recibir la señal global, cada servidor activa su propia pantalla de espera local e inicia el countdown visual de 5 segundos. Aunque cada servidor sigue mostrando su interfaz individual, ambos countdowns comienzan a partir de la misma señal coordinada, lo que garantiza un arranque prácticamente simultáneo entre las dos ventas.

Esto permite simular el caso de múltiples fechas de un mismo evento liberadas al mismo tiempo. A nivel experimental, esta etapa es importante porque demuestra que la sincronización ya no depende únicamente del estado local de cada servidor, sino de una condición global supervisada por el coordinador.

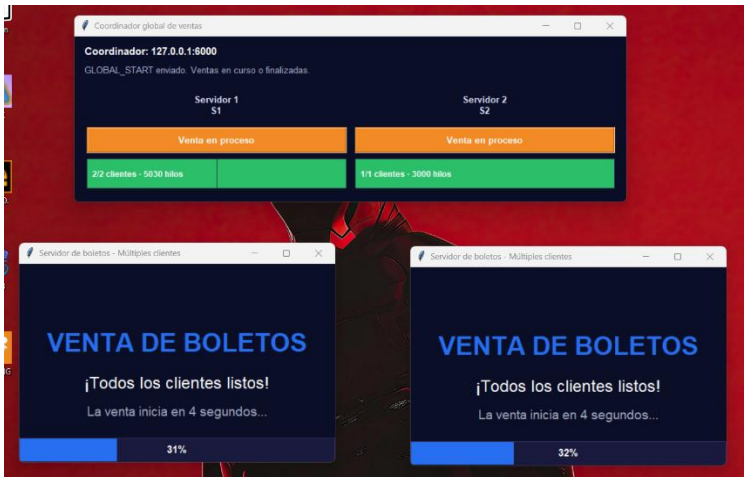


Figura 5. Countdown sincronizado en los servidores S1 y S2.

6. Inicio de venta y ejecución concurrente

Una vez concluido el countdown, cada servidor habilita la venta y libera a sus clientes locales para comenzar la compra de boletos. A partir de ahí, cada punto de acceso genera sus hilos compradores, los cuales envían solicitudes concurrentes de reserva y compra al servidor correspondiente.

En esta fase, ambos servidores operan en paralelo, pero cada uno mantiene control exclusivo sobre su propio inventario. El coordinador ya no participa en la lógica interna de compra, sino únicamente en la coordinación inicial y en la visualización del estado global del sistema.

Figura 6. Ventas activas en paralelo después del arranque sincronizado.

7. Finalización de la prueba

Finalmente, cada servidor concluye su proceso de venta de forma independiente, ya sea por agotamiento de boletos, por cierre debido a fin de clientes o por imposibilidad de vender asientos restantes según el tipo de comprador disponible. Cuando esto ocurre, el servidor cambia su estado en el coordinador a **verde**, lo que indica que su venta ha finalizado.m De forma paralela, tanto clientes como servidores imprimen sus propios resúmenes de ejecución. Esto permite contrastar el comportamiento local de cada venta con la vista global observada en el coordinador durante la prueba.

Figura 7. Estado final del coordinador con ventas concluidas.

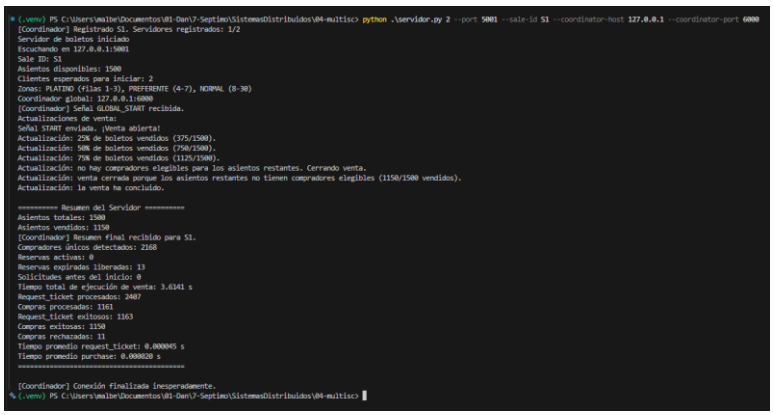
Resultados

Servidor S1

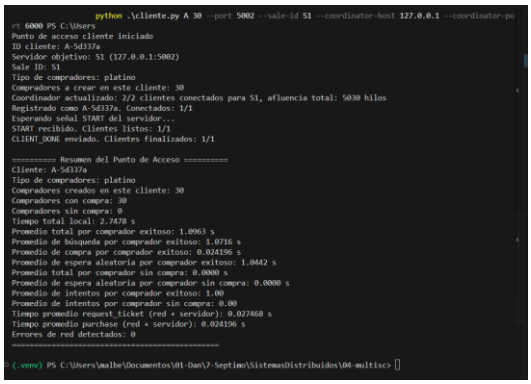
En el caso de este servidor podemos comprobar en su resumen que:

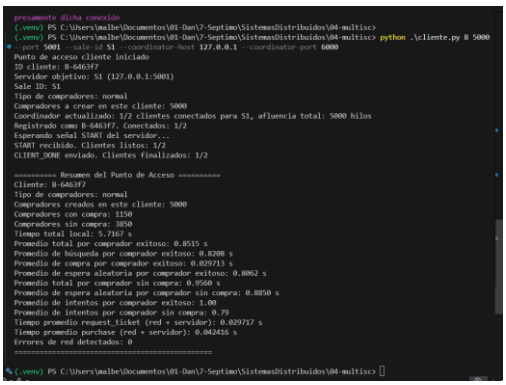
- No se agotó por inventario, se cerró por falta de demanda elegible (unsellable_remaining): vendió 1150/1500.
- Esto sugiere que quedaron principalmente asientos de zonas que los tipos de comprador activos ya no podían adquirir (por reglas de elegibilidad).
- Tuvo alta competencia:
    - 2407 solicitudes de ticket para 1163 reservas exitosas.
    - 1161 intentos de compra para 1150 compras exitosas.
- La operación fue estable:
    - 0 solicitudes antes del inicio (sincronización correcta).
    - latencias promedio muy bajas
    - solo 11 compras rechazadas.
- 13 reservas expiradas liberadas: el TTL está funcionando y evita asientos atorados.

Resumern del servidor



Clientes del servidor



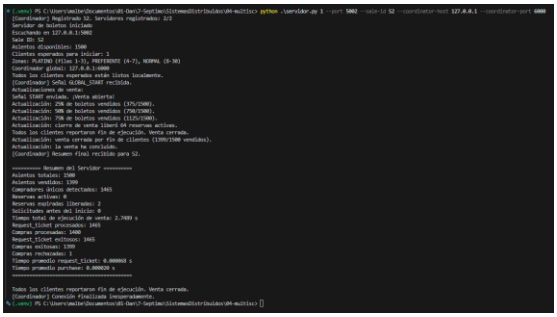


Servidor S2

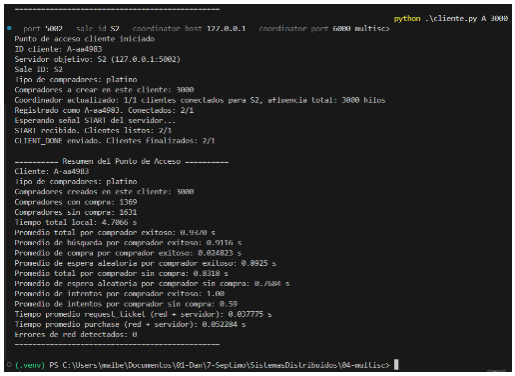
Por otro lado, para el segundo servidor tenemos algo distinto:

- Avanzó más en ocupación: 1399/1500, y cerró por fin de clientes (all_clients_done), no por falta de elegibilidad.
- Hubo una sesión muy eficiente:
  - 1465 solicitudes y 1465 reservas exitosas (100% éxito en request).
  - 1400 intentos de compra, 1399 exitosas (solo 1 rechazo).
- Al cerrar, liberó 64 reservas activas: normal cuando los clientes terminan y aún había apartados pendientes.
- También mostró buena sincronización:
  - 0 solicitudes antes del inicio.
  - tiempos promedio muy bajos

Resumen del servidor



Cliente del servidor



Comparación de resultados con prácticas anteriores

Contrastando con practicas anteriores podemos decir que la arquitectura garantiza integridad transaccional del inventario (sin sobreventa) y cierre consistente de ejecución. Sin embargo, el valor analítico principal no está en comparar tiempos absolutos entre prácticas, sino en observar cómo cambian los patrones de contención, sincronización y comportamiento bajo carga al modificar la arquitectura.

En la práctica previa de múltiples clientes y un solo servidor (Práctica 3), toda la afluencia converge en un único punto de servicio. Ese diseño permite validar con claridad la autoridad central del servidor, pero también concentra toda la competencia de sockets, bloqueos y reintentos en un solo nodo. Con 5,520 compradores (20 platino, 500 preferente y 5,000 normal) disputando 1,500 asientos, el sistema mostró una fuerte asimetría de resultados por prioridad de zona: éxito casi total en tipos altos, y alta tasa de rechazo/reintento en el tipo normal. En ese sentido, la Práctica 3 confirmó que la lógica de elegibilidad y consistencia funciona incluso bajo saturación, pero evidenció el costo de la centralización extrema: más errores de red y mayor penalización temporal en los compradores de menor prioridad.

En la práctica actual de múltiples clientes y múltiples servidores (Práctica 4), la carga ya no se concentra en una sola venta: se distribuye entre ventas simultáneas (por ejemplo, S1 y S2), y además se incorpora sincronización global de arranque mediante coordinador. Este cambio modifica la lectura de resultados: cada servidor conserva su integridad local, pero ahora el sistema agrega una capa de coordinación inter-servidor. En tus ejecuciones se observa precisamente eso:

- **S1** cerró por falta de compradores elegibles (1150/1500), lo que indica que la restricción por tipo/zona operó correctamente hasta volver invendible el remanente.
- **S2** cerró por fin de clientes (1399/1500), con mayor ocupación final y menor fricción relativa en compra.
- Ambos servidores recibieron GLOBAL_START y mantuvieron conteos consistentes, sin reservas activas al final.

Esto implica que la Práctica 4 no reemplaza la validez de la Práctica 3, sino que la extiende: la consistencia ya no solo se prueba en una venta altamente saturada, sino también en ventas paralelas coordinadas, con cierres distintos según el perfil de demanda de cada fecha.

Respecto a tiempos (**Práctica 1: 13.35 s, Práctica 2: 2.97 s, Práctica 3: 13.88 s**), se mantiene la advertencia metodológica: no son equivalentes directos por diferencias de carga, instrumentación y criterios de inicio/fin de medición. Para la Práctica 4, esa cautela es aún más importante porque ahora hay sincronización global, countdown coordinado y ejecución concurrente de servidores con distribuciones de clientes diferentes. Por ello, el criterio correcto de comparación es arquitectónico:

1. **Práctica 1 (hilos locales):** valida exclusión mutua y consistencia en memoria compartida.
2. **Práctica 2 (cliente-servidor básico):** introduce control transaccional remoto con autoridad central única, que vendría a ser el servidor, el cual maneja lock de zonas y evita el caos en la venta.
3. **Práctica 3 (múltiples clientes, un servidor):** estresa esa autoridad central con afluencia masiva y prioridades de compra.
4. **Práctica 4 (múltiples clientes, múltiples servidores y coordinador):** añade escalamiento horizontal por venta y sincronización global entre nodos, conservando consistencia local y cierre ordenado.

En síntesis, la Práctica 3 demuestra robustez de un nodo único bajo presión extrema; la Práctica 4 demuestra que esa lógica puede escalarse a múltiples ventas simultáneas sin perder integridad, separando la contención por servidor y manteniendo coordinación global de inicio. El costo operativo ya no se concentra solo en la saturación de un único servidor, sino que se distribuye por fecha y perfil de demanda, lo cual representa un paso más cercano a escenarios reales de alta concurrencia multi-evento.

Conclusiones

Esta práctica me dejó algo muy claro: cuando hay muchos usuarios intentando acceder a un mismo recurso limitado, la sincronización se vuelve algo imposible de ignorar. Aunque en esta simulación trabajé con pocos hilos comparado con un sistema real, aun así se alcanza a ver cómo la concurrencia depende del flujo de compradores, el orden en que llegan, la capacidad del sistema y las reglas que controlan el acceso.

Con esto también entendí mejor que la concurrencia no es solo tener muchos procesos ejecutándose al mismo tiempo. También implica manejar cómo cambia la demanda sobre un recurso limitado y coordinar bien los estados para evitar errores. Al final, la carga del sistema, la sincronización y la disponibilidad real de asientos están totalmente conectadas.

La idea para estas prácticas surgió de una experiencia personal: la compra de boletos para BTS hace poco más de un mes. Ese proceso fue bastante frustrante, y me dejó con curiosidad por entender qué pasa detrás de esos sistemas. La simulación me ayudó a convertir esa experiencia en un problema técnico relacionado con concurrencia, sincronización y escalabilidad.

A diferencia de la práctica anterior, aquí no solo reforcé la consistencia en el modelo cliente-servidor, sino que también trabajé con múltiples ventas ocurriendo al mismo tiempo. Me gustó haber integrado el coordinador, porque permitió observar mejor la carga en cada servidor, el estado general del sistema y el arranque sincronizado de las ventas.

Finalmente, confirmé que la integridad del sistema no se logra solo agregando más hilos, sino definiendo bien las reglas de estado, cierre y coordinación entre procesos. Con esta práctica siento que ya tengo una base más clara para avanzar a la escalabilidad de las siguientes prácticas.

Anexo

Nota: Debido a la extensión de los códigos, solo anexé captura de las primeras secciones. Los códigos completos se encuentran en la carpeta de esta entrega. código.

Código del coordinador

Código del servidor

```python
import argparse
import json
import random
import socketserver
import threading
import time
import tkinter as tk
import uuid

FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
TOTAL_COMPRADORES_ESPERADOS = TOTAL_ASIENTOS * 2
RESERVA_TTL_SEGUNDOS = 3.0


class TicketState:
    def __init__(self):
        self.state_lock = threading.Lock()
        self.terminal_lock = threading.Lock()
        self.seat_status = [["free" for _ in range(COLUMNAS)] for _ in range(FILAS)]
        self.free_seats = {(r, c) for r in range(FILAS) for c in range(COLUMNAS)}
        self.reservations = {}
        self.sold_count = 0

        self.sales_started_at = None
        self.sales_finished_at = None
        self.sold_out_event = threading.Event()
        self.sales_open_event = threading.Event()
        self.summary_printed = False

        self.history = []
        self.unique_buyers = set()
        self.hitos_reportados = set()

        self.metrics = {
            "request_ticket_count": 0,
            "purchase_count": 0,
            "request_ticket_time_total": 0.0,
            "purchase_time_total": 0.0,
            "request_ticket_ok": 0,
            "purchase_ok": 0,
            "purchase_rejected": 0,
            "expired_releases": 0,
            "not_started_count": 0,
        }

    def _log_event(self, request_id, buyer_id, action, result, seat=None, reservation_id=None, extra=None):
        event = {
            "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
            "request_id": request_id,
            "buyer_id": buyer_id,
            "action": action,
            "result": result,
            "seat": seat,
            "reservation_id": reservation_id,
        }
        if extra is not None:
            event["extra"] = extra
        self.history.append(event)

    def _cleanup_expired_locked(self):
        now = time.monotonic()
        expired = [
            reservation_id
            for reservation_id, info in self.reservations.items()
            if info["expires_at"] <= now
        ]

        for reservation_id in expired:
            info = self.reservations.pop(reservation_id)
            row, col = info["seat"]
            self.seat_status[row][col] = "free"
            self.free_seats.add((row, col))
            self.metrics["expired_releases"] += 1
            self._log_event(
                request_id=f"server-expire-{reservation_id}",
                buyer_id=info["buyer_id"],
                action="EXPIRE_RESERVATION",
                result="RELEASED",
                seat={"row": row, "col": col},
                reservation_id=reservation_id,
            )

    def open_sales(self):
        with self.state_lock:
            if self.sales_open_event.is_set():
                return
            self.sales_started_at = time.perf_counter()
            self.sales_open_event.set()

        with self.terminal_lock:
            print("Actualizaciones de venta:")

    def _report_progress_milestones_locked(self):
        for porcentaje, umbral in (
            (25, int(TOTAL_ASIENTOS * 0.25)),
            (50, int(TOTAL_ASIENTOS * 0.50)),
            (75, int(TOTAL_ASIENTOS * 0.75)),
            (100, TOTAL_ASIENTOS),
        ):
            if self.sold_count >= umbral and porcentaje not in self.hitos_reportados:
                self.hitos_reportados.add(porcentaje)
                with self.terminal_lock:
                    print(f"Actualización: {porcentaje}% de boletos vendidos ({self.sold_count}/{TOTAL_ASIENTOS}).")

    def register_buyer(self, buyer_id):
        if buyer_id is None:
            return
        with self.state_lock:
            self.unique_buyers.add(str(buyer_id))

    def sales_open(self):
        return self.sales_open_event.is_set()
```

```python
def request_ticket(self, buyer_id, request_id):
    started = time.perf_counter()

    with self.state_lock:
        if not self.sales_open_event.is_set():
            self.metrics["not_started_count"] += 1
            response = {
                "status": "not_started",
                "message": "La venta aún no inicia.",
            }
        else:
            self._cleanup_expired_locked()
            self.metrics["request_ticket_count"] += 1

            if self.sold_count >= TOTAL_ASIENTOS or not self.free_seats:
                self._log_event(request_id, buyer_id, "REQUEST_TICKET", "SOLD_OUT")
                response = {
                    "status": "sold_out",
                    "message": "No hay asientos disponibles.",
                }
            else:
                seat = random.choice(tuple(self.free_seats))
                row, col = seat
                reservation_id = str(uuid.uuid4())
                expires_at = time.monotonic() + RESERVA_TTL_SEGUNDOS

                self.free_seats.remove(seat)
                self.seat_status[row][col] = "reserved"
                self.reservations[reservation_id] = {
                    "buyer_id": buyer_id,
                    "seat": seat,
                    "expires_at": expires_at,
                }

                self.metrics["request_ticket_ok"] += 1
                self._log_event(
                    request_id,
                    buyer_id,
                    "REQUEST_TICKET",
                    "RESERVED",
                    seat={"row": row, "col": col},
                    reservation_id=reservation_id,
                )
                response = {
                    "status": "ok",
                    "reservation_id": reservation_id,
                    "seat": {"row": row, "col": col},
                    "ttl_seconds": RESERVA_TTL_SEGUNDOS,
                }

    elapsed = time.perf_counter() - started
    self.metrics["request_ticket_time_total"] += elapsed
    return response

def purchase(self, buyer_id, request_id, reservation_id):
    started = time.perf_counter()

    with self.state_lock:
        if not self.sales_open_event.is_set():
            self.metrics["not_started_count"] += 1
            response = {
                "status": "not_started",
                "message": "La venta aún no inicia.",
            }
        else:
            self._cleanup_expired_locked()
            self.metrics["purchase_count"] += 1

            if self.sold_count >= TOTAL_ASIENTOS:
                self._log_event(request_id, buyer_id, "PURCHASE", "SOLD_OUT", reservation_id=reservation_id)
                response = {
                    "status": "sold_out",
                    "message": "La venta terminó.",
                }
            else:
                info = self.reservations.pop(reservation_id)
                if info is None:
                    self.metrics["purchase_rejected"] += 1
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "TRANSACTION_BEFORE_RESERVATION",
                        reservation_id=reservation_id,
                    )
                    response = {
                        "status": "error",
                        "code": "Invalid or expired reservation",
                    }
                elif info["buyer_id"] != buyer_id:
                    self.metrics["purchase_rejected"] += 1
                    row, col = info["seat"]
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "PURCHASE",
                        seat={"row": row, "col": col},
                        reservation_id=reservation_id,
                    )
                    response = {
                        "status": "error",
                        "code": "Reserva no pertenece al comprador",
                    }
                else:
                    row, col = info["seat"]
                    self.reservations.pop(reservation_id, None)
                    self.seat_status[row][col] = "sold"
                    self.sold_count += 1
                    self._report_progress_milestones_locked()
                    self.metrics["purchase_ok"] += 1
                    self._log_event(
                        request_id,
                        buyer_id,
                        "PURCHASE",
                        "SOLD",
                        seat={"row": row, "col": col},
                        reservation_id=reservation_id,
                    )

                    if self.sold_count >= TOTAL_ASIENTOS and not self.sold_out_event.is_set():
                        self.sales_finished_at = time.perf_counter()
                        self.sold_out_event.set()

                    response = {
                        "status": "ok",
                        "seat": {"row": row, "col": col},
                        "sold_count": self.sold_count,
                        "remaining": TOTAL_ASIENTOS - self.sold_count,
                    }

    elapsed = time.perf_counter() - started
    self.metrics["purchase_time_total"] += elapsed
    return response

def get_snapshot(self):
    with self.state_lock:
        reserved_count = len(self.reservations)
        free_count = TOTAL_ASIENTOS - self.sold_count - reserved_count
        seat_status_copy = [row[:] for row in self.seat_status]
        return {
            "sold_count": self.sold_count,
            "reserved_count": reserved_count,
            "free_count": free_count,
            "seat_status": seat_status_copy,
            "buyers_counted": len(self.unique_buyers),
        }

def print_summary_once(self):
    with self.state_lock:
        if self.summary_printed:
            return
        self.summary_printed = True
    self.print_summary()
```

Código de cliente

```python
import argparse
import ison
import random
import socket
import threading
import time
import uuid

FILAS = 25
COLUMNAS = 40
TOTAL_ASIENTOS = FILAS * COLUMNAS
COMPRADORES_POR_ASIENTO = 3
TOTAL_COMPRADORES = TOTAL_ASIENTOS * COMPRADORES_POR_ASIENTO

SOCKET_TIMEOUT = 3.0


stats_lock = threading.Lock()
terminal_lock = threading.Lock()
sold_out_event = threading.Event()
threads = []

metrics = {
    "buyers_success": 0,
    "buyers_fail": 0,
    "buyer_total_success": 0.0,
    "buyer_search_success": 0.0,
    "buyer_purchase_success": 0.0,
    "buyer_wait_success": 0.0,
    "buyer_total_fail": 0.0,
    "buyer_wait_fail": 0.0,
    "request_time_total": 0.0,
    "purchase_time_total": 0.0,
    "request_count": 0,
    "purchase_count": 0,
    "attempts_success": 0,
    "attempts_fail": 0,
    "network_errors": 0,
}

sales_start_ts = None
sales_end_ts = None


def avg(total, count):
    if count == 0:
        return 0.0
    return total / count


def send_request(host, port, payload):
    data = ison.dumps(payload) + "\n".encode("utf-8")
    with socket.create_connection((host, port), timeout=SOCKET_TIMEOUT) as sock:
        sock.settimeout(SOCKET_TIMEOUT)
        sock.sendall(data)
        response_bytes = b""
        while not response_bytes.endswith(b"\n"):
            chunk = sock.recv(4096)
            if not chunk:
                break
            response_bytes += chunk

    if not response_bytes:
        raise ConnectionError("Respuesta vacía del servidor")

    return ison.loads(response_bytes.decode("utf-8").strip())


def buyer_worker(buyer_id, host, port):
    buyer_started = time.perf_counter()
    wait_random_accum = 0.0
    request_time_accum = 0.0
    purchase_time_accum = 0.0
    attempts = 0
    purchased = False

    while not sold_out_event.is_set():
        pause = random.uniform(0.1, 3.0)
        time.sleep(pause)
        wait_random_accum += pause

        request_payload = {
            "action": "request_ticket",
            "buyer_id": buyer_id,
            "request_id": str(uuid.uuid4()),
        }

        attempts += 1
        request_started = time.perf_counter()
        try:
            response = send_request(host, port, request_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        request_elapsed = time.perf_counter() - request_started
        request_time_accum += request_elapsed

        with stats_lock:
            metrics["request_count"] += 1
            metrics["request_time_total"] += request_elapsed

        request_status = response.get("status")
        if request_status == "not_started":
            time.sleep(0.2)
            continue
        if request_status == "sold_out":
            sold_out_event.set()
            break
        if request_status != "ok":
            continue

        reservation_id = response.get("reservation_id")
        if not reservation_id:
            continue

        purchase_payload = {
            "action": "purchase",
            "buyer_id": buyer_id,
            "reservation_id": reservation_id,
            "request_id": str(uuid.uuid4()),
        }

        purchase_started = time.perf_counter()
        try:
            purchase_response = send_request(host, port, purchase_payload)
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            continue

        purchase_elapsed = time.perf_counter() - purchase_started
        purchase_time_accum += purchase_elapsed

        with stats_lock:
            metrics["purchase_count"] += 1
            metrics["purchase_time_total"] += purchase_elapsed

        purchase_status = purchase_response.get("status")
        if purchase_status == "not_started":
            time.sleep(0.2)
            continue
        if purchase_status == "sold_out":
            sold_out_events.set()
            break
        if purchase_status == "ok":
            purchased = True
            sold_count = purchase_response.get("sold_count", 0)
            if sold_count >= TOTAL_ASIENTOS:
                sold_out_event.set()
            break

    buyer_total_elapsed = time.perf_counter() - buyer_started
    buyer_search_elapsed = wait_random_accum + request_time_accum

    with stats_lock:
        if purchased:
            metrics["buyers_success"] += 1
            metrics["buyer_total_success"] += buyer_total_elapsed
            metrics["buyer_search_success"] += buyer_search_elapsed
            metrics["buyer_purchase_success"] += purchase_time_accum
            metrics["buyer_wait_success"] += wait_random_accum
            metrics["attempts_success"] += attempts
        else:
            metrics["buyers_fail"] += 1
            metrics["buyer_total_fail"] += buyer_total_elapsed
            metrics["buyer_wait_fail"] += wait_random_accum
            metrics["attempts_fail"] += attempts
```

```python
def print_summary():
    success = metrics["buyers_success"]
    fail = metrics["buyers_fail"]

    with terminal_lock:
        print("\n========= Resumen de Tiempos del Cliente =========")
        print(f"Compradores creados: {TOTAL_COMPRADORES}")
        print(f"Compradores con compra: {success}")
        print(f"Compradores sin compra: {fail}")
        print(f"Tiempo total general: {sales_end_ts - sales_start_ts:.4f} s")
        print(
            f"Promedio total por comprador exitoso: "
            f"{avg(metrics['buyer_total_success'], success):.4f} s"
        )
        print(
            f"Promedio de búsqueda por comprador exitoso: "
            f"{avg(metrics['buyer_search_success'], success):.4f} s"
        )
        print(
            f"Promedio de compra por comprador exitoso: "
            f"{avg(metrics['buyer_purchase_success'], success):.6f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador exitoso: "
            f"{avg(metrics['buyer_wait_success'], success):.4f} s"
        )
        print(
            f"Promedio total por comprador sin compra: "
            f"{avg(metrics['buyer_total_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de espera aleatoria por comprador sin compra: "
            f"{avg(metrics['buyer_wait_fail'], fail):.4f} s"
        )
        print(
            f"Promedio de intentos por comprador exitoso: "
            f"{avg(metrics['attempts_success'], success):.2f}"
        )
        print(
            f"Promedio de intentos por comprador sin compra: "
            f"{avg(metrics['attempts_fail'], fail):.2f}"
        )
        print(
            f"Tiempo promedio request_ticket (red + servidor): "
            f"{avg(metrics['request_time_total'], metrics['request_count']):.6f} s"
        )
        print(
            f"Tiempo promedio purchase (red + servidor): "
            f"{avg(metrics['purchase_time_total'], metrics['purchase_count']):.6f} s"
        )
        print(f"Errores de red detectados: {metrics['network_errors']}")
        print("-------------------------------------------------------\n")


def parse_args():
    parser = argparse.ArgumentParser(description="Cliente de boletos - Práctica 2 Fase 1")
    parser.add_argument("--host", default="127.0.0.1", help="Host del servidor")
    parser.add_argument("--port", type=int, default=5000, help="Puerto del servidor")
    return parser.parse_args()


def wait_for_sales_start(host, port):
    with terminal_lock:
        print("Esperando señal de inicio del servidor...")

    while True:
        try:
            response = send_request(host, port, {"action": "health", "request_id": str(uuid.uuid4())})
        except Exception:
            with stats_lock:
                metrics["network_errors"] += 1
            time.sleep(0.4)
            continue

        if response.get("status") == "ok" and response.get("sales_open"):
            with terminal_lock:
                print("Servidor confirmó inicio de venta.")
            return

        time.sleep(0.3)


def main():
    global sales_start_ts, sales_end_ts

    args = parse_args()

    with terminal_lock:
        print("Cliente de compra iniciado")
        print(f"Servidor destino: {args.host}:{args.port}")
        print(f"Asientos esperados: {TOTAL_ASIENTOS}")
        print(f"Compradores a crear: {TOTAL_COMPRADORES}")

    wait_for_sales_start(args.host, args.port)

    sales_start_ts = time.perf_counter()

    for buyer_id in range(1, TOTAL_COMPRADORES + 1):
        thread = threading.Thread(target=buyer_worker, args=(buyer_id, args.host, args.port), daemon=False)
        threads.append(thread)
        thread.start()
        time.sleep(0.0005)

    for thread in threads:
        thread.join()

    sales_end_ts = time.perf_counter()
    print_summary()


if __name__ == "__main__":
    main()
```